

Slovenská technická univerzita v Bratislave

Fakulta informatiky a informačných technológií

Umelá inteligencia

Zadanie č. 1 – Problém obchodného cestujúceho

Optimalizácia trasy obchodného cestujúceho pomocou genetického algoritmu
a simulovaného žihania.

Sabina Matejčíková

15. október 2025

Bratislava, 2025

Úloha

Obchodný cestujúci má navštíviť viacero miest. V jeho záujme je minimalizovať cestovné náklady a cena prepravy je úmerná dĺžke cesty, preto snaží sa nájsť najkratšiu možnú cestu tak, aby každé mesto navštívil práve raz. Keďže sa nakoniec musí vrátiť do mesta z ktorého vychádza, jeho cesta je uzavretá krivka.

Zadanie

*Je daných aspoň 20 miest (20 – 40) a každé má určené súradnice ako celé čísla X a Y. Tieto súradnice sú náhodne vygenerované. (Rozmer mapy môže byť napríklad 200 * 200 km.) Cena cesty medzi dvoma mestami zodpovedá Euklidovej vzdialenosti – vypočíta sa pomocou Pytagorovej vety. Celková dĺžka trasy je daná nejakou permutáciou (poradím) miest. Cieľom je nájsť takú permutáciu (poradie), ktorá bude mať celkovú vzdialenosť čo najmenšiu.*

Riešenie

Problém obchodného cestujúceho riešime dvomi algoritmami:

- **Genetický algoritmus** – využíva selekciu, kríženie a mutáciu, na postupné zlepšovanie populácie riešení.
- **Simulované žihanie** – postupne upravuje riešenie náhodnými zmenami, pričom so znižovaním teploty obmedzuje prijímanie horších riešení a smeruje k stabilnému, lepšiemu výsledku.

Genetický algoritmus

Reprezentácia dát

Mestá sú generované náhodne pomocou funkcie `generated_cities()`. Každé mesto je reprezentované ako dvojica súradníc (x, y), ktoré určujú jeho polohu. Všetky mestá sú uložené v zozname `cities`.

```
cities = [(43, 178), (120, 34), (87, 59), (12, 199), (156, 88)]
```

Jedna možná cesta obchodného cestujúceho je reprezentovaná ako zoznam indexov miest.

```
path = [2, 0, 4, 1, 3]
```

V genetickom algoritme populácia obsahuje niekoľko jednolivcov, kde každý predstavuje jednu takúto permutáciu miest. Všetci jednotlivci sú uložený v zozname `individuals`.

```
individuals = [[0, 1, 2, 3, 4],[1, 4, 0, 3, 2],[3, 2, 4, 1, 0], ...
```

Popri zozname jednotlivcov sa vytvára aj zoznam *population*, ktorý obsahuje hodnotenie každého riešenia. Každá položka v ňom predstavuje dvojicu (*fitness*, *length*), kde:

- *length* je celková dĺžka danej cesty, vypočítaná ako súčet euklidovských vzdialeností medzi mestami,
- *fitness* je hodnota určená ako $1 / \text{length}$.

```
population = [(0.0045, 220.5),(0.0038, 262.4),(0.0051, 195.9), ...]
```

Parametre

Na začiatku sú definované základné konštanty, ktoré ovplyvňujú priebeh a správanie oboch použitých algoritmov.

WIDTH - maximálne rozmery priestoru, v ktorom sa generujú mestá.
NUM_CITIES počet miest, ktoré musí obchodný cestujúci navštíviť.
NUM_INDIVIDUALS - počet jedincov (ciest) v jednej generácii genetického algoritmu.
NUM_GENERATION - maximálny počet generácií.
NUM_LIMIT - maximálny počet nezlepšených cest.
MUTATION_RATE - pravdepodobnosť mutácie

Algoritmus

Každý jedinec predstavuje jednu náhodne vygenerovanú cestu cez všetky mestá. Cesty sú vytvárané funkciou *random_path()*, ktorá náhodne premieša poradie indexov miest.

Týmto spôsobom vznikne počiatočná populácia rôznych ciest. Každá z nich je následne vyhodnotená pomocou funkcie *calculate_fitness()*, ktorá vypočíta dĺžku cesty a jej *fitness* hodnotu. Výsledky sú uložené do zoznamu *population* vo forme dvojíc (*fitness*, *length*).

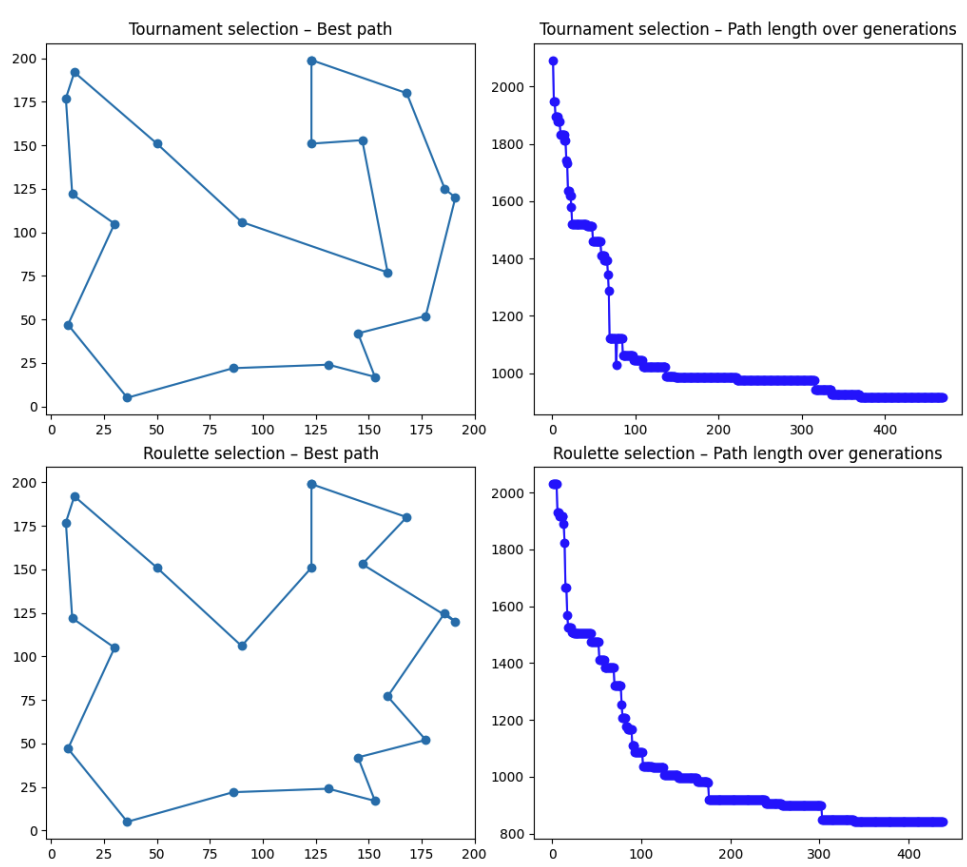
Tvorba novej generácie prebieha nasledovne:

1. Z aktuálnej populácie sa vyberú najlepšie dva jedince (elitizmus), ktoré sa prenesú priamo do novej generácie bez zmeny.
2. Zvyšní jedinci sa získavajú výberom rodičov pomocou zvolenej metódy selekcie: buď turnajovou selekciou, alebo ruletovým výberom.
3. Vybraní rodičia sa kombinujú pomocou funkcie *crossover()*, ktorá vytvára nových potomkov jednobodovým krížením.

4. Na niektorých potomkoch sa s pravdepodobnosťou `MUTATION_RATE` vykoná mutácia, pri ktorej sa náhodne vyberú dve pozície v ceste a poradie miest medzi nimi sa obráti.
5. Nová generácia sa následne vyhodnotí rovnakým spôsobom ako pôvodná — pre každého jedinca sa vypočíta dĺžka cesty a jej fitness. Najlepšie riešenie sa uloží a algoritmus pokračuje do ďalšej generácie.

Tento proces sa opakuje, až kým sa nedosiahne stanovený počet generácií alebo stabilné riešenie bez ďalšieho zlepšenia.

Zhodnotenie výsledkov a porovnanie metód



Obr. 1: Porovnanie turnajovej a ruletovej selekcie.

Pri porovnaní turnajovej a ruletovej selekcie obe metódy dosiahli podobnú kvalitu riešenia, pričom konečná dĺžka cesty sa pohybovala približne okolo hodnoty 900. Turnajová selekcia preukázala rýchlejšiu konvergenciu – už počas prvých generácií výrazne skrátila dĺžku cesty a následne sa stabilizovala. Ruletová selekcia mala pomalší, ale plynulejší priebeh, pričom si dlhšie udržiavala variabilitu populácie.

Testovanie veľkosti populácie (NUM_INDIVIDUALS)

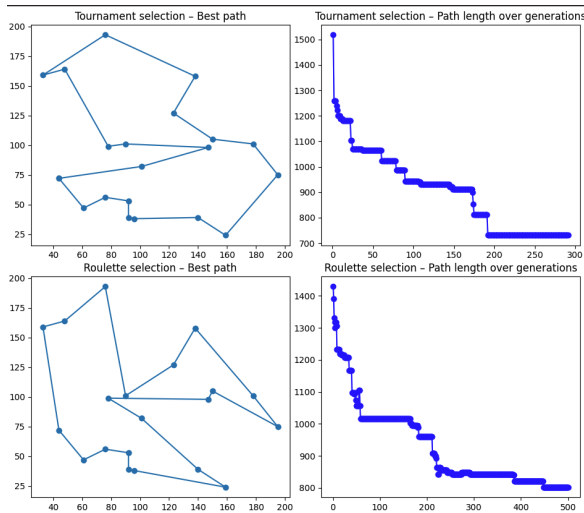
NUM_GENERATION = 500

MUTATION_RATE = 0.1

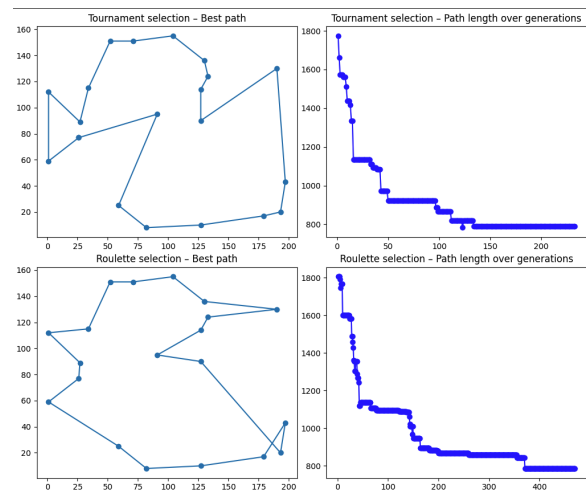
NUM_LIMIT = 100

NUM_CITIES = 20

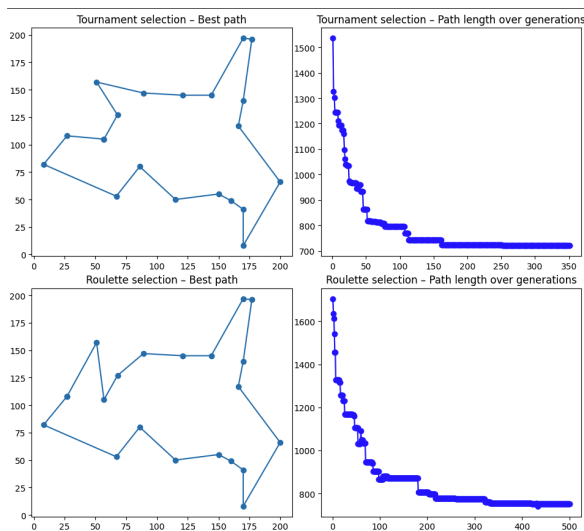
Testované hodnoty: NUM_INDIVIDUALS = 10, 20, 50, 100



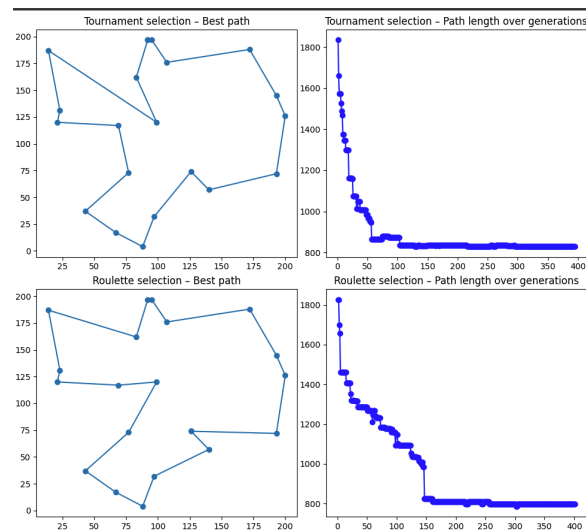
Obr. 2: Populácia 10 jedincov



Obr. 3: Populácia 20 jedincov



Obr. 4: Populácia 50 jedincov



Obr. 5: Populácia 100 jedincov

Pri malej populácii (10) algoritmus rýchlo skonvergoval, no riešenie nebolo presné a ľahko uviazol v lokálnom minime. Pri 20 jedincoch sa kvalita riešení zlepšila a priebeh bol stabilnejší. Najlepšie výsledky sa dosiahli pri 50 jedincoch, kde bol zachovaný dobrý pomer medzi presnosťou a rýchlosťou výpočtu.

Pri väčšej populácii (100) sa priebeh stal menej efektívnym — zlepšenie výsledku bolo minimálne, no výpočet trval dlhšie. Algoritmus tak spracúval priveľa podobných riešení.

Testovanie počtu generácií (NUM_GENERATION)

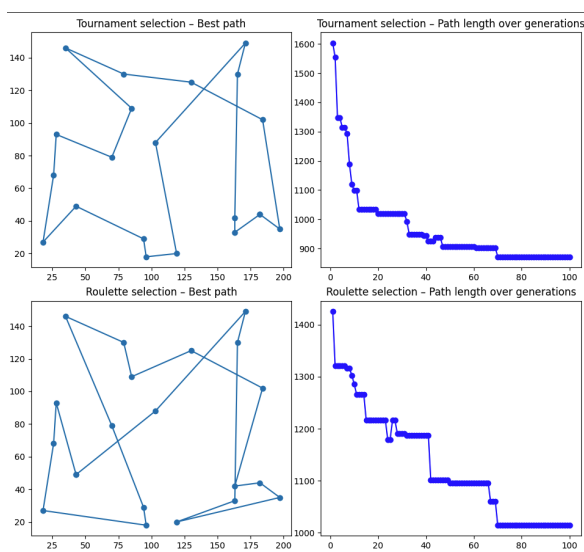
NUM_INDIVIDUALS = 50

MUTATION_RATE = 0.1

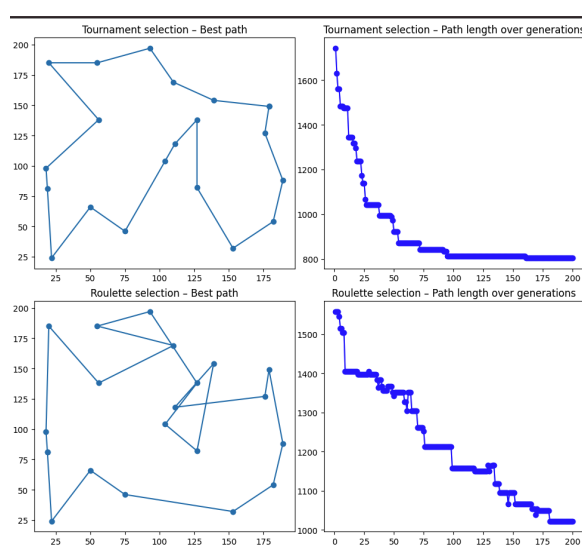
NUM_LIMIT = 100

NUM_CITIES = 20

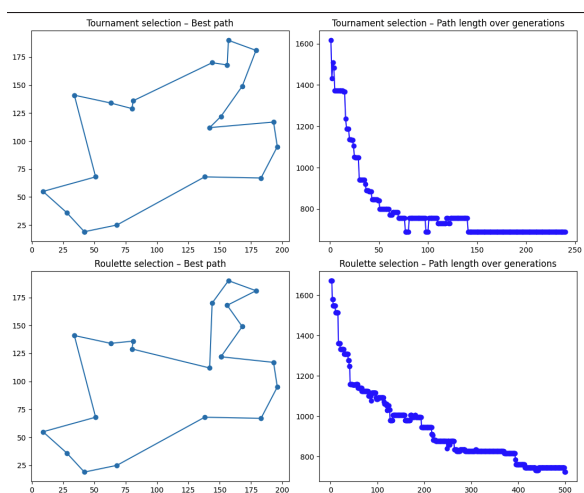
Testované hodnoty: NUM_GENERATION = 100, 200, 500



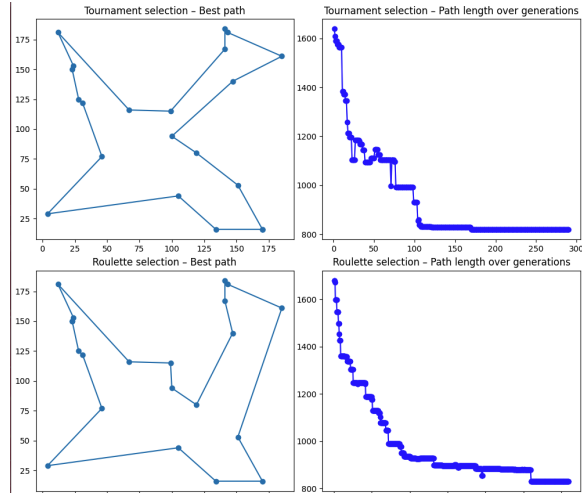
Obr. 6: Veľkosť generácie 100



Obr. 7: Veľkosť generácie 200



Obr. 8: Veľkosť generácie 500



Obr. 9: Veľkosť generácie 800

Pri 100 generáciách algoritmus skončil pomerne skoro — rýchlo sa priblížil k dobrému riešeniu, ale výsledná cesta ešte nebola optimálna. Pri 200 generáciách už bolo vidieť stabilnú konvergenciu a dĺžka cesty sa ustálila na lepšej hodnote.

Zvýšenie na 500 generácií prinieslo mierne zlepšenie, no priebeh optimalizácie bol už podobný ako pri 200. Pri 800 generáciách sa výsledky už výrazne nemenili — algoritmus len dlhšie bežal, bez podstatného zlepšenia.

Testovanie pravdepodobnosti mutácie (MUTATION_RATE)

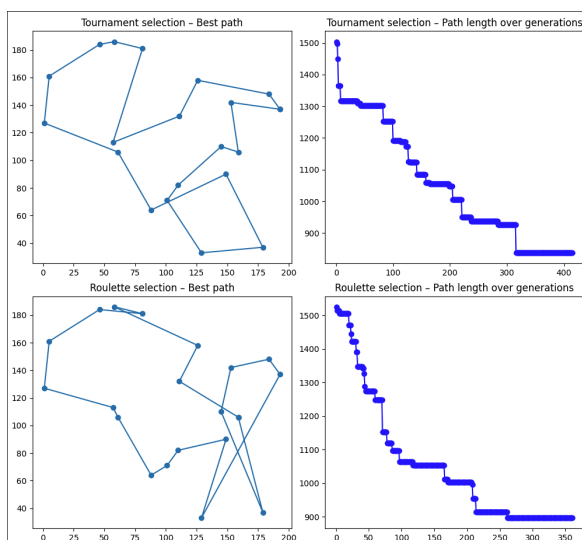
NUM_INDIVIDUALS = 50

NUM_GENERATION = 500

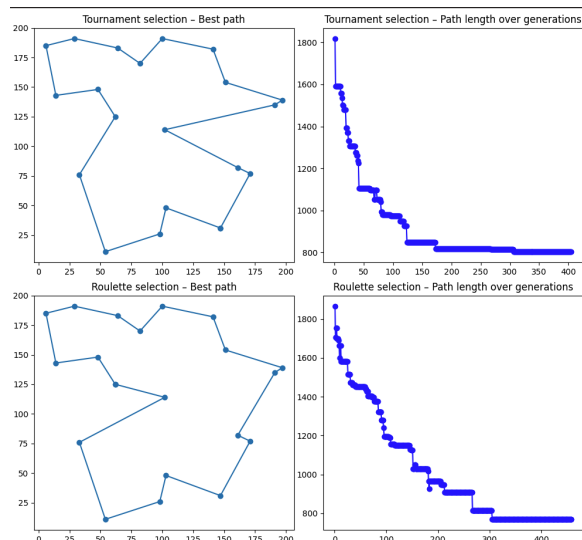
NUM_LIMIT = 100

NUM_CITIES = 20

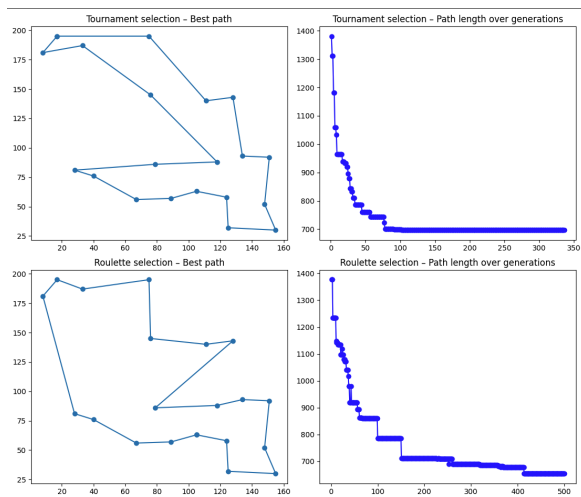
Testované hodnoty: MUTATION_RATE = 0.01, 0.05, 0.1, 0.2



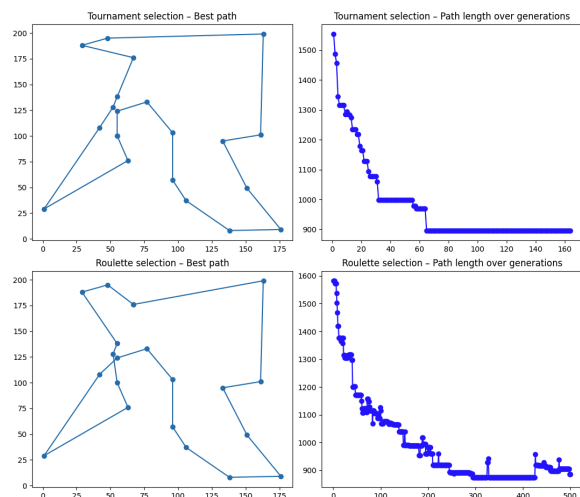
Obr. 10: Mutácia 1%



Obr. 11: Mutácia 5%



Obr. 12: Mutácia 10%



Obr. 13: Mutácia 20%

Pri veľmi nízkej hodnote (0.01) sa mutácia vyskytovala zriedka, populácia sa rýchlo zjednotila a algoritmus prestal zlepšovať riešenie. Pri 0.05 už dochádzalo k miernym zmenám v populácii a výsledky boli stabilnejšie.

Najlepšie výsledky sa dosiahli pri 0.1, kde mutácie pomáhali udržiavať rozmanitosť a zároveň umožnili postupnú optimalizáciu. Pri vysokej hodnote (0.2) už mutácie zasahovali príliš často, čo spôsobovalo, že sa algoritmus správal viac náhodne a zhoršila sa stabilita výsledkov.

Testovanie limitu generácií bez zlepšenia (NUM_LIMIT)

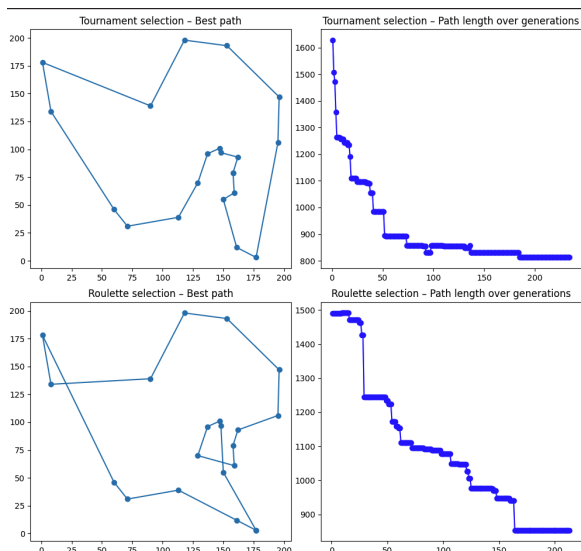
NUM_INDIVIDUALS = 50

NUM_GENERATION = 500

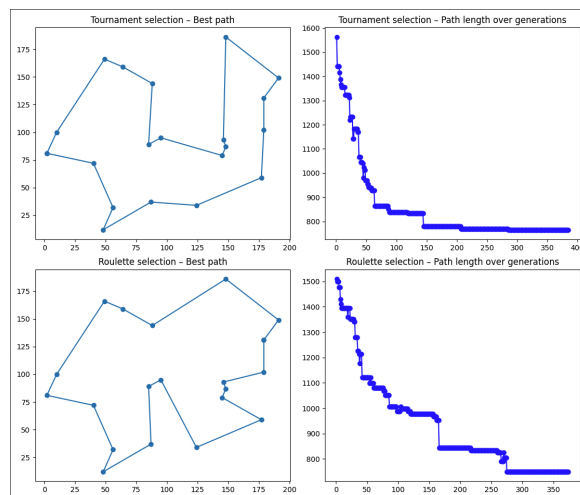
MUTATION_RATE = 0.1

NUM_CITIES = 20

Testované hodnoty: NUM_LIMIT = 50, 100



Obr. 14: Limit 50



Obr. 15: Limit 100

Pri limite 50 sa algoritmus ukončil pomerne rýchlo – optimalizácia prebehla rýchlo, no v niektorých prípadoch ešte pred dosiahnutím najlepšieho riešenia. Pri zvýšení limitu na 100 sa dosiahla lepšia kvalita výsledku, keďže algoritmus mal viac priestoru na doladenie riešenia, hoci výpočet trval dlhšie.

Simulované žihanie

Parametre

TEMPERATURE – počiatočná teplota, ktorá určuje, ako často sa akceptujú horšie riešenia.
 TEMPERATURE_RATE – rýchlosť ochladzovania,
 MIN_TEMPERATURE – minimálna teplota, pri ktorej sa proces zastaví
 WIDTH a NUM_CITIES – definujú mapu

Algoritmus

1. Inicializácia – vytvorí sa náhodné počiatočné riešenie prostredníctvom random_path.
2. Generovanie susedného riešenia – vytvorí sa nové riešenie pomocou mutate() pri ktorom sa náhodne vzberú dve pozície v ceste a poriadok miest medzi nimi sa obráti.
3. Vyhodnotenie rozdielu energie – vypočíta sa rozdiel medzi
4. Rozhodnutie o prijatí riešenia
 - Ak nové riešenie je lepšie a prijme sa.

- Ak nové riešenie nie je lepšie sa prijme s pravdepodobnosťou

$$P = e^{-\frac{\Delta E}{T}}$$

5. Zníženie teploty (chladenie) – po každom novom riešení sa teplota znižuje, aby čoraz menej prijímala horšie riešenia
6. Opakovanie – kroky 2 až 5 sa opakujú, kým teplota neklesne pod minimálnu hodnotu.

Zhodnotenie vlastností systému a vplyvu rozvrhu

Najdôležitejším faktorom je rozvrh teploty – teda spôsob, akým sa teplota v priebehu algoritmu znižuje.

Pri vysokej počiatočnej teplote algoritmus prijíma aj horšie riešenia, čo umožňuje preskakováť lokálne minimá. Ak je však teplota príliš vysoká, systém sa správa náhodne a konvergencia je pomalá. Naopak, ak sa teplota znižuje príliš rýchlo, algoritmus sa „zamrzne“ bez dostatočného prieskumu.

Hodnota parametra TEMPERATURE_RATE určuje rýchlosť ochladzovania. Pomalé ochladzovanie vedie k stabilnejšiemu, ale pomalšiemu procesu, kým rýchlejšie ochladzovanie skracuje čas výpočtu, no často znižuje kvalitu riešenia.

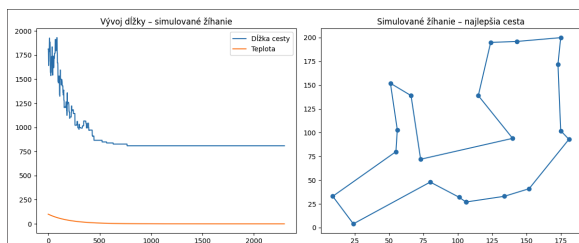
Testovanie počiatočnej teploty (TEMPERATURE)

TEMPERATURE_RATE = 0.995

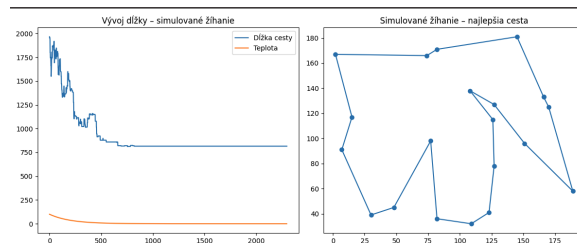
MIN_TEMPERATURE = 0.001

NUM_CITIES = 20

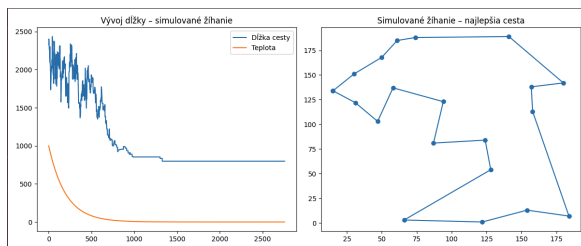
Testované hodnoty: TEMPERATURE_START = 100, 500, 1000, 2000



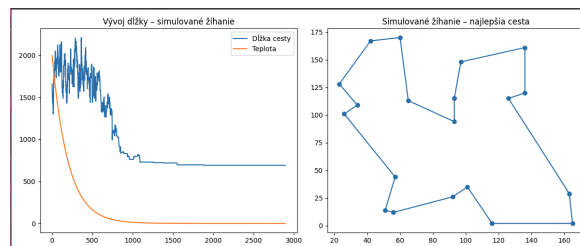
Obr. 16: Teplota 100



Obr. 17: Teplota 500



Obr. 18: Teplota 1000



Obr. 19: Teplota 5000

Pri $TEMPERATURE = 100$ sa algoritmus veľmi rýchlo ochladil a prestal prijímať horšie riešenia. To viedlo k predčasnému zastaveniu optimalizácie a k horšiemu výsledku. Pri $TEMPERATURE = 500$ sa dosiahlo mierne zlepšenie — algoritmus mal viac priestoru na preskúvanie riešení, ale stále sa pomerne rýchlo ustálil. Pri $TEMPERATURE = 1000$ sa dosiahol najlepší pomer medzi presnosťou a rýchlosťou konvergencie. Algoritmus akceptoval dostatočne veľa prechodov, vďaka čomu sa vyhol lokálnym minimám a našiel kratšiu výslednú trasu. Pri $TEMPERATURE = 5000$ sa hľadanie výrazne predĺžilo — teplota klesala pomaly a systém ostal dlhší čas v náhodnej fáze.

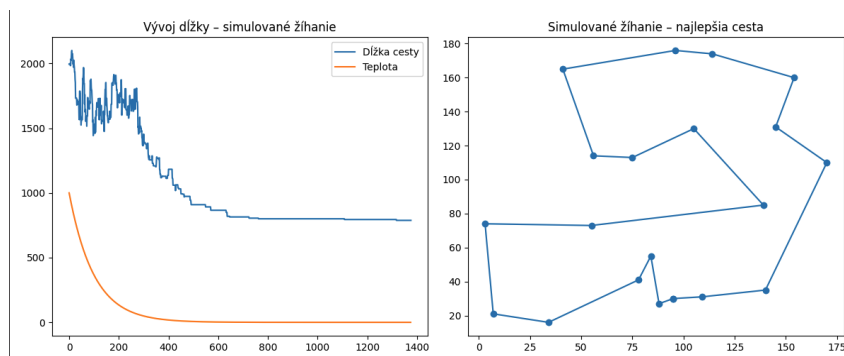
Testovanie rýchlosti ochladzovania ($TEMPERATURE_RATE$)

$TEMPERATURE_START = 1000$

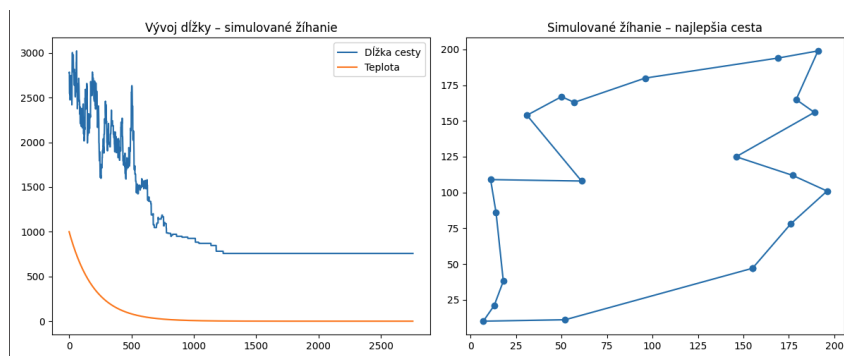
$MIN_TEMPERATURE = 0.001$

$NUM_CITIES = 20$

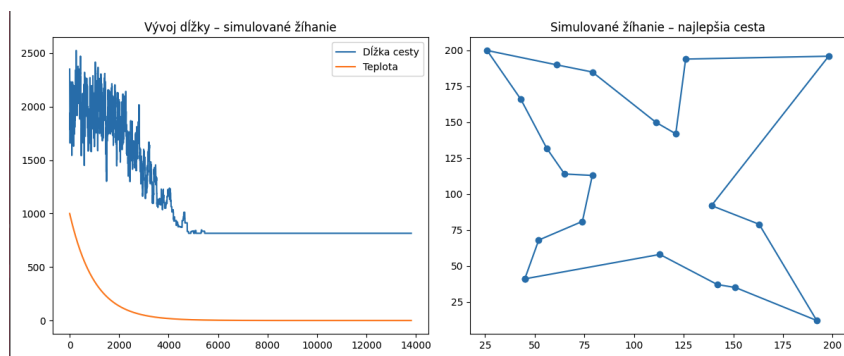
Testované hodnoty: $TEMPERATURE_RATE = 0.99, 0.995, 0.999$



Obr. 20: Rate 0.99



Obr. 21: Rate 0.995



Obr. 22: Rate 0.999

Pri $RATE = 0.99$ sa teplota znižovala rýchlo, vďaka čomu algoritmus skončil pomerne skoro. Výsledná dĺžka cesty bola však horšia, pretože systém sa ochladil skôr, než preskúmal dostatok riešení. Pri $RATE = 0.995$ sa dosiahol najlepší výsledok. Algoritmus mal vyvážený priebeh, dostatok náhodnosti na začiatku a postupne sa stabilizoval bez výrazných výkyvov. Dĺžka cesty bola najkratšia zo všetkých testov.

Pri $RATE = 0.999$ bolo ochladzovanie veľmi pomalé. Algoritmus síce preskúmal viac možností a dosiahol podobne dobré riešenie, ale výpočet trval výrazne dlhšie bez zreteľného zlepšenia oproti 0.995.

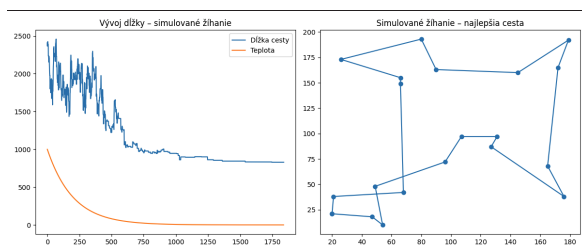
Testovanie minimálnej teploty (MIN_TEMPERATURE)

TEMPERATURE_RATE = 0.995

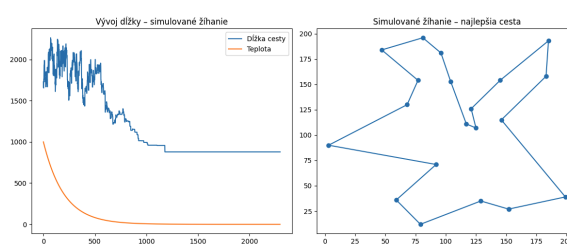
TEMPERATURE_START = 1000

NUM_CITIES = 20

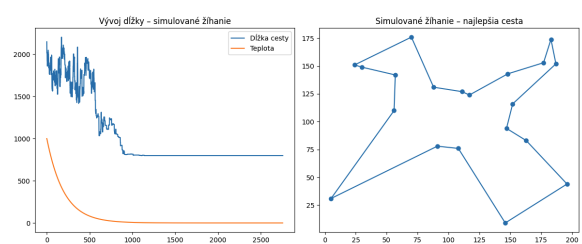
Testované hodnoty: MIN_TEMPERATURE = 0.1, 0.01, 0.001, 0.0001



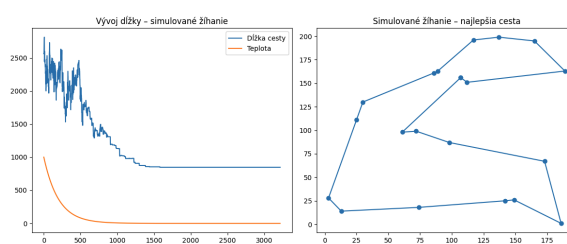
Obr. 23: Minimalna eplota 0.1



Obr. 24: Minimalna eplota 0.01



Obr. 25: Minimalna eplota 0.001



Obr. 26: Minimalna eplota 0.0001

Pri $\text{MIN_TEMPERATURE} = 0.1$ algoritmus skončil veľmi rýchlo. Teplota klesla na koncovú hodnotu už po niekoľkých iteráciách, čo viedlo k slabému výsledku a vysokej dĺžke cesty.

Pri $\text{MIN_TEMPERATURE} = 0.01$ bolo vidieť mierne zlepšenie, no proces bol stále príliš krátky a konvergencia nastala priskoro.

Pri $\text{MIN_TEMPERATURE} = 0.001$ sa priebeh stabilizoval. Algoritmus mal dostatočný čas na hľadanie optimálnejšej cesty a dosiahol veľmi dobrý výsledok.

Pri $\text{MIN_TEMPERATURE} = 0.0001$ sa dosiahol najlepšie riešenie. Ďalšie znižovanie minimálnej teploty (0.0001) už neprinieslo významné zlepšenie, iba predĺžilo čas výpočtu. —

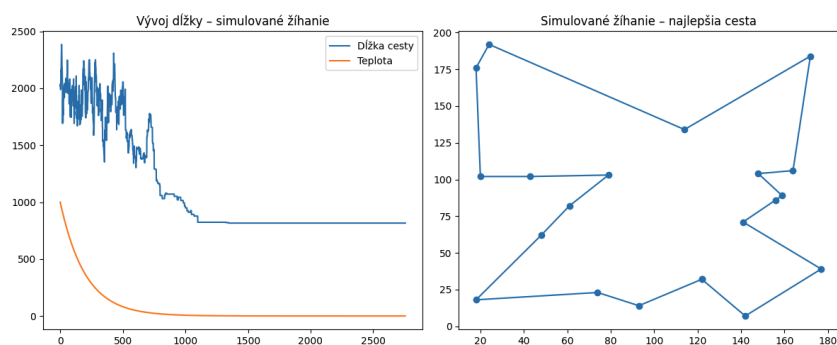
Testovanie počtu miest (NUM_CITIES)

`TEMPERATURE_START = 1000`

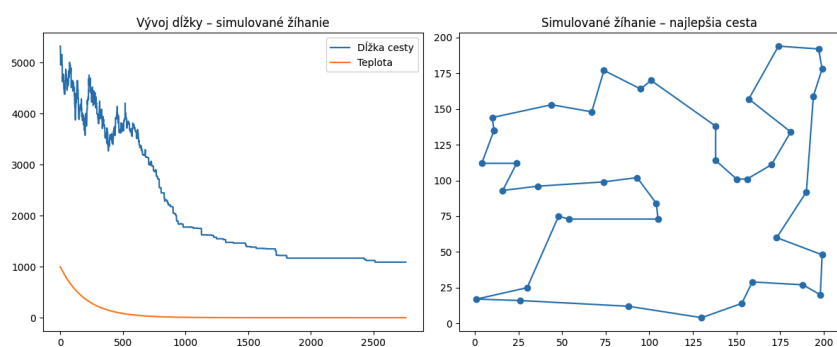
`MIN_TEMPERATURE = 0.001`

`TEMPERATURE_RATE = 0.995`

Testované hodnoty: `NUM_CITIES = 20, 40`



Obr. 27: Počet miest 20



Obr. 28: Počet miest 40

Pri 20 mestách algoritmus veľmi rýchlo dosiahol stabilný priebeh. Po krátkej dobe cesta prudko klesla a následne sa ustálila. Výsledok predstavuje takmer optimálnu cestu s krátkou dĺžkou a minimálnym počtom výkyvov.

Pri 40 mestách bol priebeh dlhší a menej stabilný. Na začiatku sa dĺžka cesty výrazne menila a algoritmus potreboval viac krokov, kým sa dostal k uspokojivému riešeniu. Napriek tomu bolo pozorované postupné zlepšovanie a stabilizácia výsledku po dostatočne dlhom čase.

Zhodnotenie a porovnanie algoritmov

V projekte boli implementované a analyzované dva prístupy k riešeniu problému obchodného cestujúceho — genetický algoritmus a simulované žihanie.

Genetický algoritmus

Genetický algoritmus pracuje s populáciou riešení, ktoré sa v priebehu generácií kombinujú a mutujú. Jeho výhodou je paralelné prehľadávanie viacerých oblastí riešenia a schopnosť dostať sa mimo lokálnych miním vďaka mutáciám a kríženiu. Správne zvolená veľkosť populácie a miera mutácie výrazne ovplyvňujú rýchlosť aj kvalitu výsledkov. Pri testovaní sa ukázalo, že

turnajová selekcia je stabilnejšia a vedie k rýchlejšej konvergencii, zatiaľ čo ruletová selekcia je náchylnejšia na výkyvy, ale dokáže niekedy nájsť lepšie riešenia po dlhšom čase.

Simulované žihanie

Simulované žihanie predstavuje jednoduchší, ale veľmi účinný prístup. Na rozdiel od genetického algoritmu pracuje len s jedným riešením, ktoré postupne vylepšuje. Kľúčovým parametrom je rozvrh teploty — teda ako rýchlo sa systém „ochladzuje“. Ak teplota klesá príliš rýchlo, algoritmus sa zastaví v lokálnom minime; naopak, pomalé ochladzovanie vedie k lepšiemu riešeniu, ale za cenu dlhšieho výpočtu. Pri vhodnom nastavení sa dosahujú veľmi kvalitné výsledky, často porovnateľné s genetickým algoritmom.

Záver

Cieľom práce bolo analyzovať a porovnať dva optimalizačné prístupy na riešenie problému obchodného cestujúceho – genetický algoritmus a simulované žíhanie.

Genetický algoritmus preukázal svoju silu vďaka paralelnému prehľadávaniu viacerých oblastí riešenia. Správne nastavenie parametrov umožňuje dosiahnuť kvalitné riešenia s prijateľným časom výpočtu.

Simulované žíhanie zaujalo svojou jednoduchosťou a schopnosťou postupne vylepšovať jedno riešenie. Aj napriek tomu, že algoritmus pracuje sekvenčne, jeho výkon závisí najmä od vhodne zvoleného rozvrhu teploty.

Porovnaním oboch metód možno konštatovať, že genetický algoritmus dosahuje rýchlejšiu konvergenciu a je stabilnejší pri väčších množinách dát, kým simulované žíhanie je výpočtovo menej náročné a jednoduchšie na implementáciu.